



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Profesor Oscar Reyes
Pruebas de Software (INF331)



Departamento de Informática
Universidad Técnica Federico Santa María

Proyecto de Pruebas de software

Entrega 1, Equipo 4

Joaquín De Ferrari, Javier Maturana y Rodrigo Tallar

Video: <https://youtu.be/6ZlYeoBmBRU>



Índice

1. Introducción.....	3
2. Alcances de la herramienta.....	3
a. Requerimientos funcionales.....	3
b. Requerimientos no funcionales.....	4
3. Descripción del trabajo realizado.....	4
4. Estrategia de pruebas utilizadas.....	8
5. Procedimiento de ejecución de pruebas.....	12
6. Reporte de resultados.....	13
7. Problemas encontrados y soluciones.....	14



1. Introducción

El presente trabajo trató sobre el desarrollo de una aplicación web de remates en línea, donde clientes pueden realizar pujas y un administrador puede agregar productos/subastas.

Se realizó el proyecto en 2 partes principales, un repositorio para el front-end (<https://github.com/Pruebas-intensas/Proyecto-Front>) en donde se trabajó con Angular para el desarrollo del software y Karma / Jasmine para los tests, y otro repositorio para el back-end (<https://github.com/Pruebas-intensas/Proyecto-Back>) que se trabajó con Node.js / Express para el desarrollo y Mocha para las pruebas. Para la base de datos se usó Postgres montado sobre un docker..

Todos estos servicios fueron hosteados en el computador de uno de los compañeros del grupo, usando Nolo como proveedor del dominio y dns dinámico, permitiendo conectarse al frontend con el puerto 80, al backend con el puerto 3000, a la base de datos en el puerto 5432 y a pgadmin en el puerto 8080, en la url "proyectoremates.hopto.org".

2. Alcances de la herramienta

a. Requerimientos funcionales

Búsqueda de productos: El sistema debe permitir a los usuarios autenticados acceder a un catálogo de productos a partir de un parámetro de búsqueda.

Información detallada: Los usuarios deben poder acceder a información detallada sobre los productos en subasta. Esta información debe incluir descripción, imágenes, precio actual, historial de pujas, tiempo restante de la subasta y cualquier otro detalle relevante.

Medición en tiempo real de los oferentes: El sistema debe mostrar quiénes están participando activamente en las subastas, actualizando en tiempo real sus pujas recientes y mostrando las últimas pujas realizadas.

Interfaz de administración: El administrador del sitio web debe tener acceso a una interfaz de administración sencilla. Esta interfaz debe permitir la gestión de usuarios y gestión de subastas/productos.

Registro e inicio de sesión: Los usuarios deben poder registrarse en el sitio web proporcionando su nombre, dirección de correo electrónico y contraseña. El sistema debe validar la dirección de correo electrónico y garantizar que no haya duplicados en la base de datos. Además, los usuarios registrados deben poder iniciar sesión en sus cuentas utilizando su dirección de correo electrónico y contraseña.

Realizar pujas/ofertas: Los usuarios registrados deben poder realizar pujas u ofertas en las subastas activas. Para hacerlo, deben ingresar la cantidad que



desean ofertar, y el sistema debe registrar y actualizar las ofertas en tiempo real. Además, se deben establecer reglas claras para la realización de ofertas, como incrementos mínimos de puja, y el sistema debe garantizar que las ofertas cumplan con estas reglas.

b. Requerimientos no funcionales

Seguridad de los datos: Mantener los datos de los usuario como nombre, métodos de pago y dirección de envío encriptados.

Rapidez en la carga de datos: Permitir una carga rápida a los clientes del sitio previniendo el uso alto de recursos para evitar consumir sus datos.

3. Descripción del trabajo realizado

Para esta primera entrega, en base a los requerimientos planteados, se decidió que se harían 5 vistas básicas:

-Inicio de sesión: Los usuarios ingresan con su correo y contraseña. En la misma vista se muestra el link para registrarse.

El rincón del olvido

Sistema de remates y subastas

Ingresar correo...

contraseña

Iniciar sesión

[Crear cuenta](#)



-Registro de usuario: Usuario ingresa correo, nombre, apellido y contraseña, tiene un botón aceptar que crea al usuario en base de datos.

Registro

Armando

Paredes

Correo Electrónico

Se requiere un correo

...

Repetir Contraseña

La contraseña debe tener por lo menos 6 caracteres.

Registrar Cuenta

[¿Ya tienes una cuenta? ¡inicia sesión!](#)

-Home de usuario/búsqueda de productos: Una vez que el usuario inicia sesión, puede ver los productos/remates publicados en la página que aún estén vigentes (no han pasado su fecha de término), junto a una descripción y el precio actualizado según la puja más alta. Si el usuario presiona sobre uno de los productos, se redirige a la vista de detalles de ese producto.

Productos disponibles		
Patito de Hule Pack de Patitos de Hule ideales para la hora del baño, son flotantes, al apretarlos chillan. Material: Goma de... Precio actual: 1.203 CLP	Tanque Soviético El T-34 es un tanque mediano de fabricación soviética que fue producido entre 1940 y 1958. El... Precio actual: 200.000 CLP	Balon de oro Balon de oro original firmado por Messi, Cristiano, Benzema y Modric Precio actual: 1.000.000 CLP
Cable HDMI Cable HDMI en perfecto estado, tiene un par de pines doblados pero funciona bien Precio actual: 100 CLP	Jugo Zuko Un jugo de muy buena calidad, poco uso. Precio actual: 11.000 CLP	Cuatrimoto Infantil 49 cc Spider Azul Si quieres vivir grandes aventuras todo terreno, la adrenalina y la velocidad, junto a tus hijos, este cuatrimoto es... Precio actual: 11.500 CLP



-Detalle de producto: Muestra los datos del producto particular, se puede ver la descripción completa junto a su precio inicial y su precio actual (actualizado en tiempo real). Es el lugar donde los usuarios pueden realizar pujas/ofertas, que actualizan el precio actual del producto.

Producto: Tanque Soviético

Descripción

El T-34 es un tanque mediano de fabricación soviética que fue producido entre 1940 y 1958. El desarrollo del T-34 partió de la serie de tanques rápidos BT, y Christie M1928 con blindaje inclinado, con la intención de reemplazar al tanque de caballería BT y al tanque de infantería T-26. Tras el poco éxito de los tanques soviéticos en la Guerra Civil Española, la falta de blindaje se hizo notable. El T-34 equilibraba potencia de fuego, movilidad y blindaje, aunque al inicio su eficacia en el campo de batalla fue malograda debido a la mala disposición ergonómica del compartimiento de la tripulación, la carencia de radios, la falta de municiones y el empleo de tácticas pobres, así como una anticuada cadena de mando. A finales de 1943 se introdujo el mejorado T-34/85, con un cañón de mayor potencia. El diseño y construcción del tanque fueron continuamente refinados durante la guerra para mejorar su eficacia y reducir los costes, permitiendo que se dispusiera de un número de tanques cada vez mayor. Hacia 1945, el versátil y rentable T-34 había sustituido a la mayoría de los tanques pesados y ligeros en servicio. Fue una influencia en el desarrollo posterior del nuevo concepto de tanque, el tanque de combate principal.

Precio inicial	Precio actual
\$200000	\$200000

Hacer oferta

200000

Aceptar

-Administración de productos: Muestra todos los productos existentes en la plataforma, junto a sus datos relevantes. Se agrega una columna que permite ver fácilmente si el producto efectivamente fue comprado antes de su fecha de término, y se muestra en rojo cuando ya ha pasado la fecha de término de un producto. Posee además botones para agregar y editar productos, que abren un modal donde se pueden ingresar sus datos, y un botón eliminar.

Administración de Productos

Aquí puede ver los productos de la página, modificarlos, agregar nuevos o eliminar.

Agregar Producto

Nombre	Precio Mínimo	Última Oferta	Fecha de Término	Comprado?	Descripción	
Patito de Hule	\$1203	\$	27/10/2023	No	Pack de Patitos de Hule ideales para la hora del b...	EditarEliminar
Neumáticos Dunlop	\$12000	\$	03/10/2023	No	La compañía de neumáticos Dunlop fue fundada...	EditarEliminar
Tanque Soviético	\$200000	\$	28/10/2023	No	El T-34 es un tanque mediano de fabricación sovi...	EditarEliminar
Balon de oro	\$1000000	\$	21/10/2023	No	Balon de oro original firmado por Messi, Cristian...	EditarEliminar
Cable HDMI	\$100	\$	28/10/2023	No	Cable HDMI en perfecto estado, tiene un par de ...	EditarEliminar
Jugo Zuko	\$1000	\$11000	13/10/2023	No	Un jugo de muy buena calidad, poco uso.	EditarEliminar
Televisor Samsung	\$2000	\$15000	06/10/2023	No	Especificaciones principales Tecnología: LED Con...	EditarEliminar
Cuátrimoto Infantil 49 cc Spider Azul	\$10000	\$11500	13/10/2023	No	Si quieres vivir grandes aventuras todo terreno, l...	EditarEliminar



Organización para el desarrollo:

Se utilizó una variante de Gitflow para la gestión de los cambios, que implicó la creación de ramas de features cada vez que se quisiera agregar nueva funcionalidad al software. Los cambios sobre estas ramas eran probados localmente por cada uno de los integrantes del grupo. Sin embargo, la base de datos siempre fue compartida durante el desarrollo. Por esta vez, muchos de estos cambios fueron llevados directamente a la rama main y testeados en ella (mediante pruebas automáticas y manuales). Sólo hacia el final del desarrollo se estableció que se utilizaría una rama development sobre la que se reunirían un conjunto significativo de cambios antes de hacer un release que sería testado sobre los servidores de hosting. Una vez pasadas las pruebas, la rama release sería llevada a master.

En paralelo, se utilizó Jira para definir tareas a realizar por cada integrante. Se utilizó de forma bastante básica, en modalidad Kanban, teniendo las columnas de “Por Hacer”, “En Curso” y “Listo”. Las tareas iban siendo movidas en la medida en que cada integrante cumplía sus tareas asignadas.

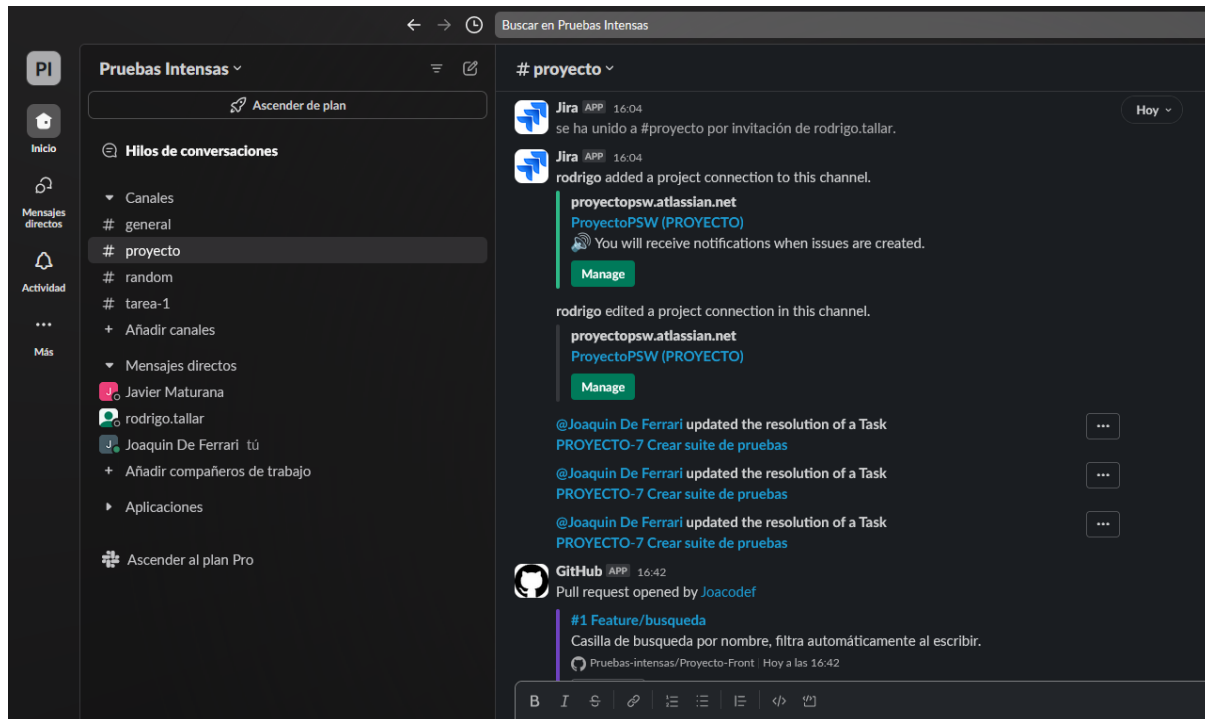
Proyectos / ProyectoPSW

Tablero PROYECTO

JF R JM

POR HACER 2	EN CURSO 1	LISTO 13 ✓
[usuario] Hacer búsqueda de productos ✓ PROYECTO-10 Hacer que se encripten contraseñas, asegurar que no se vean datos en el sitio ✓ PROYECTO-12 + Crear incidencia	Crear suite de pruebas ✓ PROYECTO-7 JM	[usuario] Hacer vista de detalle de productos ✓ PROYECTO-11 ✓ R Implementar jasmine en el front ✓ PROYECTO-16 ✓ R Implementar mocha chai en el backend ✓ PROYECTO-17 ✓ JF Agregar campo de fecha término a subastas ✓ PROYECTO-15 ✓ JF Investigar cómo usar mocha con angular ✓ PROYECTO-2 ✓ JM

Por último, se utilizó Slack conectado a Github y Jira, donde se recibían notificaciones de todas las acciones realizadas en ambos. Aún así, se recurrió frecuentemente a Discord como medio de comunicación, por lo que el uso de slack fue parcial.



4. Estrategia de pruebas utilizadas

Se utilizó una combinación de pruebas manuales y automáticas de caja negra. La estrategia consistió en realizar las pruebas automáticas en paralelo al desarrollo, para verificar si los cambios realizados “rompían” algunas de las funcionalidades previamente realizadas. Una vez realizado un conjunto de cambios significativos, se realizaban pruebas manuales para validar con mayor detalle funciones y posibles problemas específicos para la funcionalidad agregada.

La siguiente es la suite de pruebas utilizada, tanto para pruebas automáticas como para pruebas manuales.



ID	Entrada	Resultado esperado	Resultado obtenido	Fallo o Éxito	Comentarios adicionales
1	En la vista de administración de productos, agregar datos en el modal de agregar producto, seguido de presionar "Aceptar".	Se agrega un nuevo producto con los datos especificados en la lista general de productos	El esperado.	Éxito	Prueba manual front-end
2	Presionar el botón "Eliminar" en la vista de administrador, para algún producto en particular.	El producto se elimina de la tabla.	El esperado.	Éxito	Prueba manual front-end
3	En la vista de administración, seleccionar un producto y cambiar su descripción	Descripción de producto se actualiza	El esperado	Éxito	Prueba manual front-end
4	Colocar el usuario y el correo para entrar a la página	Se accede a la página	El esperado	Éxito	Prueba manual front-end
5	En la vista del usuario hacer una oferta por un monto mayor al actual	La oferta se actualiza	El esperado.	Éxito	Prueba manual front-end
6	En la vista del usuario hacer una oferta por un monto igual al actual	La oferta no se realiza	La oferta se realiza	Fallo	Prueba manual front-end
7	En la vista del usuario hacer una oferta por un monto menor al actual	La oferta no se realiza	La oferta se realiza	Fallo	Prueba manual front-end
8	En la vista de login, ingresar con un usuario válido	Se puede ingresar	Se puede ingresar	Éxito	Prueba automatizada front-end
9	En la vista de login, ingresar con un usuario invalido	No se puede ingresar	No se puede ingresar	Fallo	Prueba automatizada front-end
10	En la vista del login, hacer clic en ingresar sin rellenar ningún campo	No se puede ingresar	No se puede ingresar	Éxito	Prueba automatizada front-end
11	En la vista del registro, rellenar los campos usando un nombre menor a 3 caracteres	No se puede registrar	No se puede registrar	Éxito	Prueba automatizada front-end
12	En la vista del registro, rellenar los campos usando una contraseña menor a 6 caracteres	No se puede registrar	No se puede registrar	Éxito	Prueba automatizada front-end
13	En la vista del registro, rellenar los campos usando un nombre mayor a 3 caracteres y contraseña mayor a 6	Se puede registrar	Se puede registrar	Éxito	Prueba automatizada front-end
14	Request POST a la ruta usuario, ingresando nombre, correo y contraseña en el body.	Se recibe un código 201 y un objeto con el campo "id".	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
15	Request GET a la ruta usuario/all	Se recibe un código 200 y un arreglo.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
16	Request GET a la ruta usuario,	Se puede recibir un	El	Éxito	Prueba



	entregando como parámetro de query id=1	código 404 o 200, en el segundo se debe recibir un objeto con campo id=1.	esperado.		automatizada back-end(con Mocha-Chai)
17	Request PUT a la ruta usuario, entregando como parámetro de query id=1, y un body con nombre, password y correo, todos con valor="modificado". Esto debe recibir un id, que luego se utiliza para hacer un GET a usuario con ese id.	Se recibe código 200, y un objeto con el id especificado, además de nombre, password y correo con valor "modificado".	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
18	Request DELETE a la ruta usuario, entregando como parámetro el id recibido en la prueba anterior (usuario creado para pruebas). Luego se realiza un GET con ese id.	Se recibe código 200 en la primera parte y código 404 en la segunda.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
19	Request POST a la ruta admin, ingresando nombre, correo y contraseña en el body.	Se recibe un código 201 y un objeto con el campo "id".	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
20	Request GET a la ruta admin/all	Se recibe un código 200 y un arreglo.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
21	Request GET a la ruta admin, entregando como parámetro de query id=1	Se puede recibir un código 404 o 200, en el segundo se debe recibir un objeto con campo id=1.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
22	Request PUT a la ruta admin, entregando como parámetro de query id=1, y un body con nombre, password y correo, todos con valor="modificado". Esto debe recibir un id, que luego se utiliza para hacer un GET a admin con ese id.	Se recibe código 200, y un objeto con el id especificado, además de nombre, password y correo con valor "modificado".	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
23	Request DELETE a la ruta admin, entregando como parámetro el id recibido en la prueba anterior (usuario creado para pruebas). Luego se realiza un GET con ese id.	Se recibe código 200 en la primera parte y código 404 en la segunda.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
24	Request POST a la ruta producto, ingresando nombre = 'test', precio_minimo=100, descripcion='test' y fecha_termino='2021-06-01'.	Se recibe un código 201 y un objeto con el campo "id".	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
25	Request GET a la ruta producto/all	Se recibe un código 200 y un arreglo.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
26	Request GET a la ruta producto, entregando como parámetro de query	Se puede recibir un código 404 o 200, en	El esperado.	Éxito	Prueba automatizada



	id=1	el segundo se debe recibir un objeto con campo id=1.			back-end(con Mocha-Chai)
27	Request PUT a la ruta producto, entregando como parámetro de query id=1, y un body con nombre='modificado', precio_minimo=3000, descripcion='modificado' y fecha_termino='2021-06-02'. Esto debe recibir un id, que luego se utiliza para hacer un GET a usuario con ese id.	Se recibe código 200, y un objeto con el id y los campos especificados en el input.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
28	Request DELETE a la ruta producto, entregando como parámetro el id recibido en la prueba anterior (usuario creado para pruebas). Luego se realiza un GET con ese id.	Se recibe código 200 en la primera parte y código 404 en la segunda.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
29	Request POST a la ruta oferta, ingresando monto en el body.	Se recibe un código 201 y un objeto con el campo "id".	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
30	Request GET a la ruta oferta/all	Se recibe un código 200 y un arreglo.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
31	Request GET a la ruta oferta, entregando como parámetro de query id=1	Se puede recibir un código 404 o 200, en el segundo se debe recibir un objeto con campo id=1.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
32	Request PUT a la ruta oferta, entregando como parámetro de query id=1, y un body con monto con valor 2000. Esto debe recibir un id, que luego se utiliza para hacer un GET a oferta con ese id.	Se recibe código 200, y un objeto con el id especificado, además de nombre, password y correo con valor "modificado".	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)
33	Request DELETE a la ruta oferta, entregando como parámetro el id recibido en la prueba anterior (usuario creado para pruebas). Luego se realiza un GET con ese id.	Se recibe código 200 en la primera parte y código 404 en la segunda.	El esperado.	Éxito	Prueba automatizada back-end(con Mocha-Chai)

5. Procedimiento de ejecución de pruebas

Para el frontend se utilizó karma/jasmine, ya que es la herramienta que viene por defecto en Angular.js.

```
32
33     it('should detect form is valid', () => {
34         fixture.nativeElement.querySelector('button').click();
35         expect(component.registroForm.valid).toBeFalsy();
36     });
37
38     it('should validate correct user and password', () => {
39         component.registroForm = FormBuilder.group({
40             email: 'rtallax@gmail.com',
41             password: '123'
42         })
43         fixture.nativeElement.querySelector('button').click();
44         expect(component.registroForm.valid).toBeTruthy();
45     });
46
47     it('should validate min length password', () => {
48         component.registroForm.setValue({
49             nombre: 'Pedro',
50             apellido: 'Perez',
51             email: 'pedro@gmail.com',
52             password: '123',
53             cnfPass: '123'
54         });
55         fixture.nativeElement.querySelector('button').click();
56         expect(component.registroForm.valid).toBeFalsy();
57     });
58
59     it('should validate min length nombre', () => {
60         component.registroForm.setValue({
61             nombre: 'Po',
62             apellido: 'Perez',
63             email: 'pedro@gmail.com',
64             password: '123456',
65             cnfPass: '123456'
66         });
67         fixture.nativeElement.querySelector('button').click();
68         expect(component.registroForm.valid).toBeFalsy();
69     });
70
```

Luego de haber codificado las pruebas en el front-end, para correrlas simplemente se debe usar el comando *ng test*, que automáticamente ejecuta todas las pruebas definidas en los distintos archivos y muestra un resumen con el estado de cada prueba.

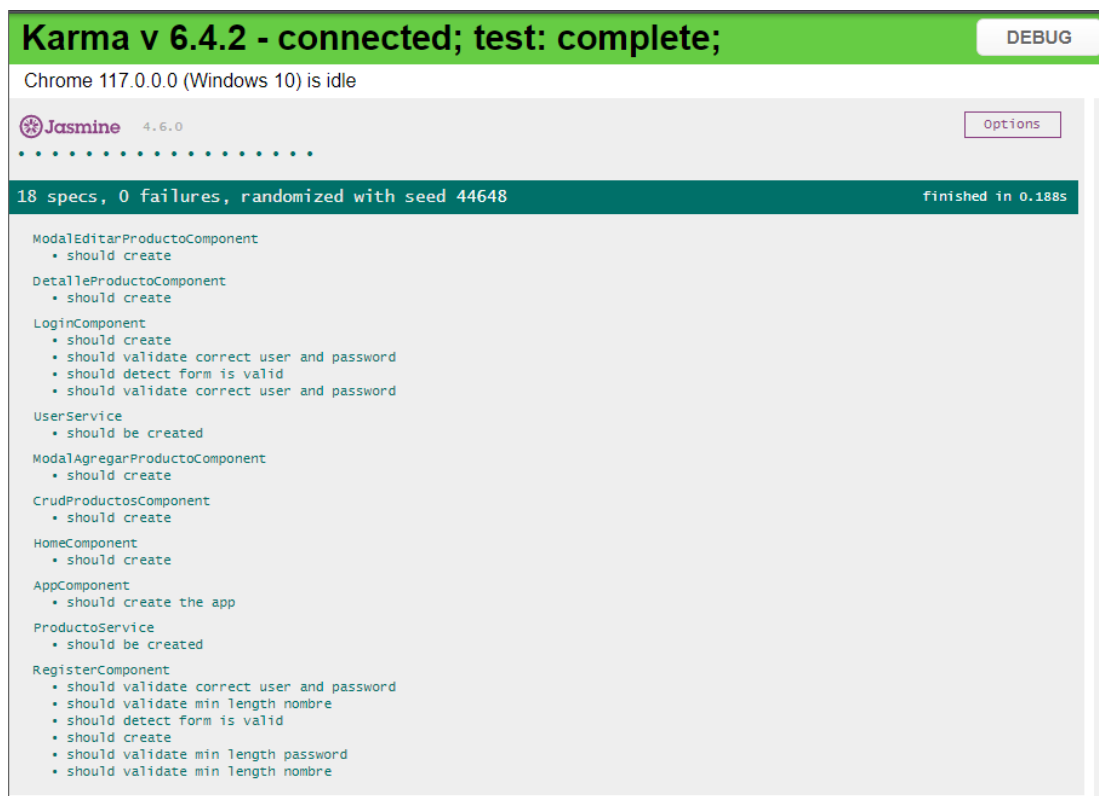
Para las pruebas del backend, por otro lado, se utilizó Mocha + Chai, las que son fácilmente implementables para testear backends que utilicen REST, dada la facilidad para crear requests y recibir sus respuestas.

```
5 const chai = require('chai');
6 const expect = require('chai').expect;
7
8 chai.use(chaiHttp);
9 chai.should();
10
11 describe('Test de obtener un admin', () => {
12   describe('GET admin?id=1', () => {
13     it('GET 1 debería retornar 200 o 404, si es 200 debería retornar un body con un objeto', (done) => {
14       chai.request(app)
15         .get('/admin?id=1')
16         .end((err, res) => {
17
18           expect(res.status).to.be.oneOf([200, 404]);
19           if(res.status == 200){
20             res.body.should.be.an('object');
21             res.body.should.have.property('id');
22             res.body.id.should.be.eq(1);
23           }
24           done();
25         });
26     });
27   });
28 });
```

Con esto, para las 4 tablas creadas en este proyecto (admin, usuario, oferta y producto), se crearon pruebas que testean la correcta realización de operaciones CRUD. Esto se utiliza realizando npm test en el backend de Express.js.

6. Reporte de resultados

Pruebas realizadas con Jasmine / Karma para el front-end



Karma v 6.4.2 - connected; test: complete; DEBUG

Chrome 117.0.0.0 (Windows 10) is idle

Jasmine 4.6.0 Options

18 specs, 0 failures, randomized with seed 44648 finished in 0.188s

- ModalEditarProductoComponent
 - should create
- DetalleProductoComponent
 - should create
- LoginComponent
 - should create
 - should validate correct user and password
 - should detect form is valid
 - should validate correct user and password
- UserService
 - should be created
- ModalAgregarProductoComponent
 - should create
- CrudProductosComponent
 - should create
- HomeComponent
 - should create
- AppComponent
 - should create the app
- ProductoService
 - should be created
- RegisterComponent
 - should validate correct user and password
 - should validate min length nombre
 - should detect form is valid
 - should create
 - should validate min length password
 - should validate min length nombre



Resultados de algunas de las pruebas realizadas con Mocha + Chai:

```
✓ POST debería retornar 201 y retornar un body con un objeto y propiedad id
Executing (default): SELECT "producto"."id", "producto"."nombre", "producto"."precio_minimo", "producto"."descripcion", "producto"."fecha_termino", "ofertas"."id" AS "ofertas.id", "ofertas"."monto" AS "ofertas.monto", "ofertas"."id_usuario" AS "ofertas.id_usuario", "ofertas"."id_producto" AS "ofertas.id_producto" FROM "producto" AS "producto" LEFT OUTER JOIN "oferta" AS "ofertas" ON "producto"."id" = "ofertas"."id_producto" WHERE "producto"."id" = '22';
✓ GET 1 (despues de POST) debería retornar 200 y retornar un body con un objeto y id igual al creado
Executing (default): UPDATE "producto" SET "nombre"=$1,"precio_minimo"=$2,"descripcion"=$3,"fecha_termino"=$4 WHERE "id" = $5
✓ GET producto recién modificado (0.0485ms)
✓ PUT debería retornar 200 y retornar un body con un objeto y propiedad id (1011ms)
Executing (default): DELETE FROM "producto" WHERE "id" = '22'
✓ DELETE debería retornar 200 y retornar un body con un objeto y propiedad id
✓ GET producto recién eliminado (0.1044ms)
Executing (default): SELECT "id", "correo", "password", "nombre" FROM "usuario" AS "usuario" WHERE "usuario"."id" = '1';
✓ GET 1 debería retornar 200 o 404, si es 200 debería retornar un body con un objeto
Executing (default): SELECT "id", "correo", "password", "nombre" FROM "usuario" AS "usuario";
✓ GET all debería retornar 200 y retornar un body con un arreglo
Executing (default): INSERT INTO "usuario" ("id","correo","password","nombre") VALUES (DEFAULT,$1,$2,$3) RETURNING "id","correo","password","nombre";
✓ POST debería retornar 201 y retornar un body con un objeto y propiedad id
Executing (default): SELECT "id", "correo", "password", "nombre" FROM "usuario" AS "usuario" WHERE "usuario"."id" = '47';
✓ GET 1 (despues de POST) debería retornar 200 y retornar un body con un objeto y id igual al creado
Executing (default): UPDATE "usuario" SET "nombre"=$1,"password"=$2,"correo"=$3 WHERE "id" = $4
✓ GET usuario recién modificado (0.0354ms)
✓ PUT debería retornar 200 y retornar un body con un objeto y propiedad id (1020ms)
Executing (default): DELETE FROM "usuario" WHERE "id" = '47'
✓ DELETE debería retornar 200 y retornar un body con un objeto y propiedad id
✓ GET usuario recién eliminado (0.0781ms)

24 passing (4s)
```

7. Problemas encontrados y soluciones

Con respecto al testing, no sabíamos usar las herramientas elegidas previamente por lo que nos demoramos una considerable cantidad de tiempo en poder implementar pruebas 'básicas' de forma automatizadas y aun así no logramos automatizar toda la suite de pruebas que teníamos preparada, obligándonos a realizar algunas de forma manual.

Por otra parte, una de las pruebas automatizadas que realizamos '*should validate min length password*' descubrió un error que existía en el software. Al realizar el testeo esta prueba falló cuando pensamos que debía pasar, al revisar el código se comprobó que la restricción del largo mínimo se estaba haciendo sobre el correo y no la contraseña, una vez que esto fue arreglado en el código la prueba pasó.

En cuanto al desarrollo del software hubo más que nada problemas con la cantidad de tiempo que pudimos dedicarle, mayormente debido a la cantidad de tiempo consumida en otros ramos. Esto se vio reflejado en varios componentes, por ejemplo, no alcanzamos a implementar el login del todo, por lo que no existe forma de ingresar como administrador y para acceder a la vista hay que navegar directamente a la url. Problema similar ocurre con los usuarios comunes, cuando se hace clic sobre iniciar sesion, se manda una request al back-end para revisar las



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Profesor Oscar Reyes
Pruebas de Software (INF331)



Departamento de Informática
Universidad Técnica Federico Santa María

credenciales y luego se redirige a la vista *home*, pero si uno navega hacia la url se puede saltar el login y ver directamente los productos (no existe una protección de la vista).